

Unity / C# / DOTs · 게임 클라이언트 개발자

정우태

아이디어를 넘어 완성까지,
개발자 정우태입니다.

QUICK VIEW

간략하게 보기 — 웹 요약본

5분 요약·플레이 영상은 웹에서. nulma.github.io/#projects

Web

nulma.github.io

GitHub

github.com/NulMa

Email

jungwt524@gmail.com

CONTENTS

목차

01	프로젝트 맵 5개 프로젝트 한눈에	04	02	Ghost March Google Play 출시 · DOTS 재설계 · 트러블슈팅	05-06
03	Match3LogueLike AI 협업 경계 설계 · 검증 루프	07-08	04	IronSarcophagus VR 협동 · 팀 프로젝트 책임 경계	09
05	LostCone 공개 개발 방향 전환 · 입력 트러블슈팅	10-11	06	Cubika 작게 완성 · itch.io WebGL 배포	12
07	기타 구현 · 작업 방식 Dong_Mae · MyTokenMate · 공통 작업 기준	13-14	08	링크 / 연락처 영상 · 플레이 링크 · 연락처	15

ABOUT

소개

C# · Unity 기반 게임 클라이언트 개발자입니다. 출시 · 구조 설계 · 트러블슈팅을 직접 경험하며, 다른 장르의 문법을 녹여 차별점을 만드는 방식으로 작업합니다.

이 령 · 교 육

학 령 건국대학교 글로벌캠퍼스 컴퓨터공학과 · 2017-2023

병 역 육군 병장 만기진역 · 2018.04-2019.11

수 상 2023 총청권 게임 인디유 공모전 장려상

부 트 캠프 Unity 부트캠프 6개월 과정 수료 · 2026.05

자 격 증

Unity Certified Associate: Game Developer · 2026.05

OPIc IM2 · 2022.11

기 술 스택

C#

Unity

DOTS

ShaderLab

HLSL

UGUI

Input System

DoTween

Unity MCP

PROJECT MAP

프로젝트 맵



1인 개발 · Google Play 출시

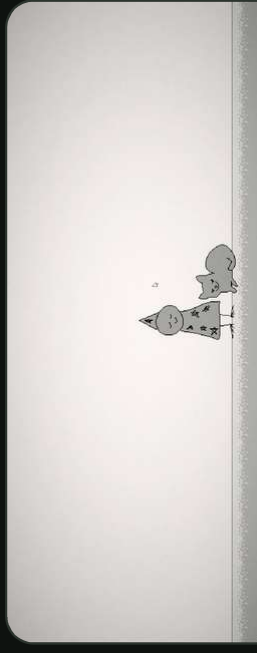
Ghost March

한국 무속 테마 모바일 survival-like. 출시 후 DOTS 수직 슬라이스로 대량적 처리 성능을 재설계.

출시

DOTS

운영



공개 개발 · 팬게임

Lost Cone

팬층 대상 플랫폼포머. 유저 설문-플레이테스트로 방향을 바꾼 공개 개발 사이클을 경험.

공개개발

피드백



1인 개발 · AI 협업 구조

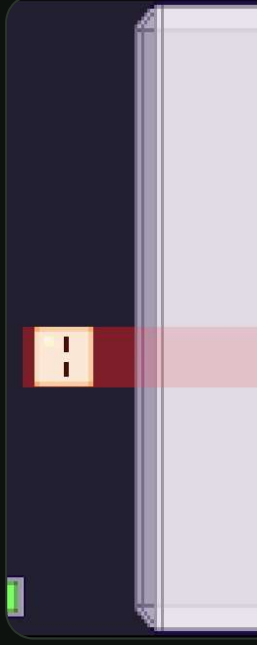
Match3LogueLike

모바일 매치3 로그라이크. AI가 구조를 깨지 않도록 모듈경제·금지 규칙·테스트를 먼저 설계.

asmdef

AI 협업

테스트



1인 개발 · WebGL

Cubika

정사각 큐브 병합 퍼즐. itch.io WebGL 배포, 아이디어부터 완성까지 1주 내 솔로 제작.

WebGL

퍼즐

솔로



2인 팀 · VR

IronSarcophagus

VR 전차 협동 게임. 포수 조준·탄도 계산·적 AI 노출 판정을 본인 책임 경계로 담당.

VR

팀

AI

GHOST MARCH · 01

출시작을 기준으로 남기고 다시 설계하다

장르 루프를 끝까지 완성해 **Google Play 출시**, 이후 출시본을 기준으로 두고 DOTS
로 대량 적 처리를 재설계했습니다.

한국 무속 테마 모바일 survival-like. 구조적 병목을 분리해 같은 장면에서 성능을 재검증하는 방향으로 이어갔습니다.



▶ **PLAY VIDEO**

플레이 영상

youtu.be/E25iW69OW6k ↗

Google Play 출시

ShaderLab · C# · HLSL · UGUI

개발 목적

작은 범위라도 완성·수상·출시까지 가
저가며 모바일 클라이언트의 전체 루프
를 경험.

핵심 구성

웨이브, 경험치/레벨업, 무기·특수기 커
맨드, 보스, 저장, 일시정지, UI 피드백.

장르 차별화

리듬·격투 게임의 커맨드 입력을 공극
기 게이지 수급과 연결해 뱀사라이크에
변주.

운영까지 문서로 — 단순 출시에서 끝내지 않음

AAB release, Billing 권한, App Update, localization, AdMob을 release checklist로 관리. 민감 정보(keystore, ad id, 인증서)
는 코드·문서에서 분리해 노출하지 않음.

195

C# 파일

72

운영·구현 문서

GHOST MARCH · 02

성능과 트러블슈팅에서 남긴 근거

문제를 측정하고 구조를 바꿔 차이를 수치로 남겼습니다. 수치는 "구조 변경이 왜 필요했는가"의 근거입니다.

7.3x

CPU 프레임타임 단축

109.5ms → 14.92ms · 4096 enemies 67fps

출시본을 기준으로 두고 DOTS 실험본과 같은 장면에서 비교.

691 → 4

렌더 batch — instancing으로 GPU/CPU 절감

9fps

Legacy 1500 stress scene 병목 기준점

트러블슈팅 — 적 과밀집을 흐름으로 해결

과밀집 →
총돌 무시 관통

FlowField로
밀집 자체 방지

경로 난잡
2차 문제

반시계 보정값
흐름 안정화

해법은 문제를 단순화해서 도입했습니다 — "플레이어 눈에 난잡하다 → 난잡함을 없애야 한다 → 내가 흐름을 정리하자." 코드 트릭이 아니라 관점 정리에서 나온 결정이었습니다.

프로파일러로 찾은 진짜 원인

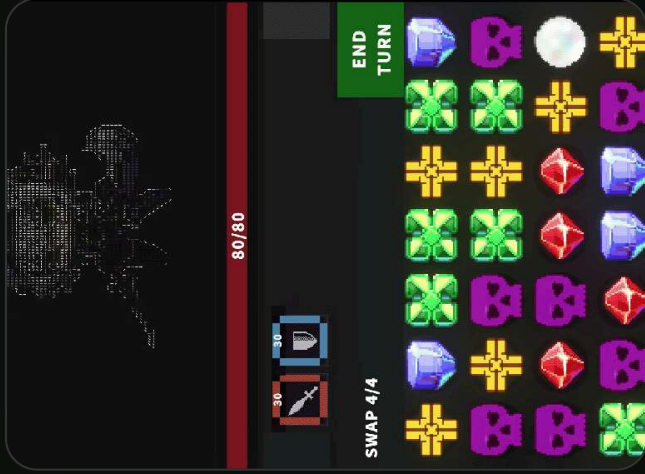
DOTS 전환 중 스파이크를 추적하다, 적 프리팸마다 카메라가 하나씩 달려 있던 문제를 발견했습니다. 수치가 아니라 실제 객체 구성을 본 덕에 잡은 케이스로, 이후엔 "코드만 보지 말고 씬 구성을 의심"하는 점검을 루틴에 넣었습니다.

남은 리스크 — 솔직하게, 계획과 함께

데미지 테스트가 과다해지면 프로파일러 스파이크가 발생. 큰 데미지만 표기하거나 군집을 그 리드로 나눠 표시 수를 제한하는 방식을 검토 중입니다.

얻은 것

출시 코드를 "끝난 코드"로 두지 않고 기존본 실험본을 분리 비교하면, 구조 변경의 효과와 위험을 동시에 볼 수 있다는 점. 성능 개선은 기술 교체보다 기존 장면·측정 항목·되돌릴 수 있는 범위를 먼저 정하는 일이 중요했습니다.



▶ PLAY VIDEO

플레이 영상

youtu.be/mGgBsoj-2JA ↗

MATCH3LOGUELIKE · 01

AI가 건드릴 수 있는 경계를 먼저 설계하다

목적은 AI로 코드를 많이 만드는 게 아니라, **AI가 작업해도 구조가 깨지지 않게** 경계·금지 규칙·테스트를 먼저 세우는 것이었습니다.

모바일 portrait 매치3 로그라이크 — 1인 개발, AI 협업 구조 실험.

모듈 레이어 — 책임 분리

Match3.Data · SO / interface / enum, MonoBehaviour 금지

Match3.Core · EventBus, 공통 런타임 규칙

Board / Combat / Roguelike / Meta

UI / Ads / Tests

경계를 지키는 규칙

- Board와 Combat은 직접 참조하지 않고 **EventBus**로만 통신.
- 원격 JSON → 로컬 캐시 → ScriptableObject 기|본값 순서로 데이터 fallback.
- 생성 코드가 영향을 줄 범위를 **asmdef.test assembly**로 제한.

AI 워크플로우 — 만드는 것보다 "검증·실패"를 설계

작업 = **Codex** / 검증 = **Claude Code**로 분리해, AI 결과의 신뢰 범위를 역할로 나눴습니다.

Unity **MCP** 연결 + 컨텍스트 압축 오픈소스 토큰을 점검하고, 스킴을 전체관리자·UI·레벨·문서로 분리.

11

asmdef 경계

34

test 파일

8

작업 agent

6

위험 패턴 hook

MATCH3LOGUELIKE · 02

검증 루프와 남은 리스크를 분리

기능 테스트가 통과해도 실제 모바일 화면에서 읽히지 않을 수 있습니다. 그래서 문서·테스트 근거·화면 QA를 다른 레이어로 분리했습니다.

STEP 1

설계

GDD·데이터 우선순위·모듈 경계·금지 규칙을 먼저. 벗어난 제안은 구현 전 걸러냄.

STEP 2

생성

AI 작업은 정해진 lane·asmdef 안에서만. Data layer엔 MonoBehaviour 참조 불가.

STEP 3

검증

test-architecture scan·merge preview·git diff를 분리. 통과 기준을 하나로 합치지 않음.

STEP 4

리뷰

스크린샷·화면 QA로 실제 플레이 화면의 문제를 다시 확인. 보상창·터치 영역·작은 텍스트.

테스트·QA를 다른 레이어로

RewardOverlayTests 10/10, SaveManagerPhase1SmokeTests 4/4 등 단위 테스트와 통합 smoke test를 나눠, 실패했을 때 어느 레이어가 깨졌는지 먼저 보게 했습니다. Simulator 화면에선 가독성·터치 영역·보상 overlay를 별도로 기록 — "기능 통과"와 "화면에서 사용 가능"을 분리했습니다.

아트 정체성 — AI 이질감을 ASCII로

개인 작업에서 AI 생성 아트는 이질감이 큼니다. 이를 역이용해 **생성 이미지를 ASCII 아트로 변환**하는 파이썬 스크립트를 시도, 보스·오브젝트의 톤을 하나로 묶었습니다. 보드 위 ASCII 보스가 그 결과물입니다.

얻은 것

AI 협업에서 중요한 것은 "AI가 할 수 있는 일"보다 "AI가 하면 안 되는 일을 막는 구조"였습니다. 생성 결과를 만기 전에 경계·금지 규칙·검증 결과를 서로 다른 문서와 테스트로 남겨야 했습니다.



▶ TEAM DEMO

팀 프로젝트 영상

youtu.be/l2X7-3QnWks ↗

협업 방식

팀원 시스템과 맞닿는 지점을 **이벤트·인터페이스**로 나뉘, 같은 파일을 동시에 만지지 않도록 경계를 분리했습니다.

IRONSARCOPHAGUS · 팀 프로젝트

팀 프로젝트에서 많은 시스템 경계

본인 담당은 **포수 조준·탄도 계산·적 AI·전차 상태 연결**, 팀원 시스템과는 **이벤트·인터페이스**로 경계를 나눴습니다.

"협동의 불편함이 재미가 된다"는 컨셉의 VR 전차 시뮬레이션 · 2인 팀.

본인 담당

- 네트워크 기반 구조와 게임플레이 모드 정리.
- 포수 VR 조준, **BallisticCalc**, **FireSystem** 구현.
- 적 AI 노출도 판정 + NavMesh 기반 재배치 후보 탐색,
- TankStatus와 피해 판정 흐름 연결.

시스템 흐름

AimController · 포수 조준 입력

BallisticCalc · 포물선 / TimeOfFlight

FireSystem · 발사체 / Linecast / Impact

TankStatus · 부위 HP / 관통 / 파괴

5-point

노출도 측정점

20%

차폐 판정 기준

12방향

재배치 후보 탐색

7

경계 asmdef

얻은 것

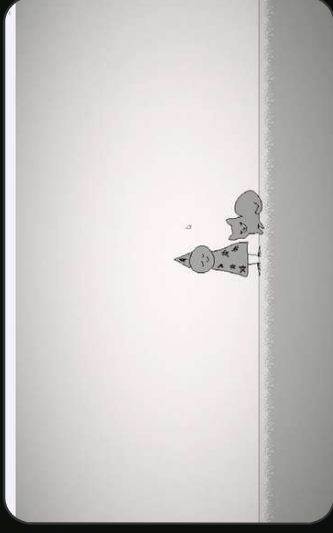
팀 프로젝트에서는 "많이 구현했다"보다 "어디까지가 내 책임인지, 어떤 이벤트·인터페이스로 연결되는지"를 먼저 정리해야 출력이 들어옵니다.

LOSTCONE · 01

공개 개발에서 방향을 바꾼 팬게임

유저 설문·플레이테스트를 근거로, 복잡한 퍼즐형 메트로베니아에서 세계관·탐험 중심으로 방향을 바꿨습니다.

팬층 대상 플랫폼포머 · 개발일지 50회의 공개 개발.



▶ PLAY VIDEO

플레이 영상

youtu.be/6QyYSIKdJPw ↗

공개 개발 기록 — 개발일지 50회로 진행 상황을 커뮤니티에 공개.

단점을 인지하고 데이터로 교정

"한 분야에 집중하면 시야가 매몰된다"는 약점을 알기에, 무의식적 가정(유저가 게임에 익숙할 것)을 커뮤니티 설문·플레이테스트로 검증해 탐험·힐링 방향으로 조정했습니다.

STEP 1

공개 기록

개발일지로 진행을 공개하고 반응을 수집.

STEP 2

테스트

설문·플레이테스트로 조작·난이도·길찾기 이해도 확인.

STEP 3

방향 전환

복잡한 퍼즐보다 이스터에그·탐색을 강화.

STEP 4

구현 반영

컷씬·fade-아이템 이동, 저장 기능을 감성 유지에 맞춰 다듬음.

Aseprite · 리소스 직접 제작

DoTween · 프롤로그 / fade-아이템 연출

얻은 것

내가 만들고 싶은 구조보다 **실제 플레이할 사람의 이해도와 기대**를 먼저 봐야 한다는 점을, 공개 개발 과정에서 데이터로 확인했습니다.

LOSTCONE · 02

간헐적 입력 중단을 코드 밖 상태까지 추적

같은 코드·같은 커밋에서도 키보드 입력이 중단되는 문제였습니다. 코드 변경만으로는 원인을 잡을 수 없었고, 실행 환경의 **Input System** 상태를 직접 확인하며 원인을 분리했습니다.

증상

플레이 중 키보드 입력이 간헐적으로 끊김. 저장 데이터·씬 전환 문제처럼 보였습니다.

실패한 시도

ActionMap 강제 활성화, Domain Reload, Library 삭제, git checkout을 차례로 확인했지만 증상 유지.

원인

터치스크린 노트북에서 Input System 이 Control Scheme을 **Touch로 auto-switch**. 코드 diff가 아니라 실행 환경 상태 문제였습니다.

해결

Keyboard&Mouse scheme 강제 ·prefab default scheme 변경으로 입력 장치를 고정.

결정적 단서 — 엔진 상태를 직접 보다

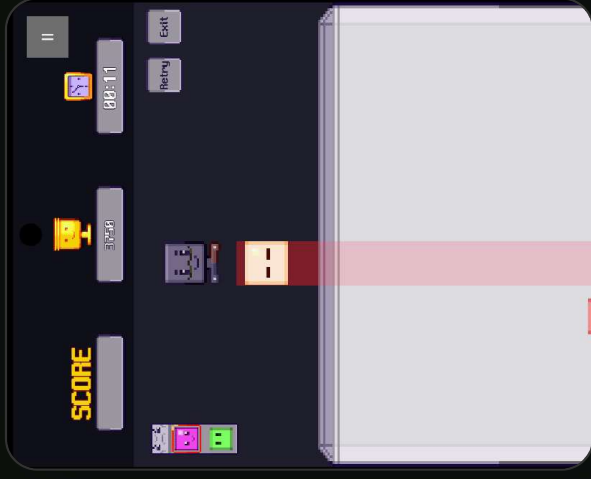
코드만 보면 ActionMap·Update timing·scene load 순서를 의심하게 됩니다. 하지만 단서는 **엔진 캡처에서 현재 연결된 장치와 선택된 scheme**이었고, 이를 시에게 보여주며 좁혀 찾았습니다. "코드만 되돌리면 해결된다"는 가정이 오히려 추적을 늦췄습니다.

다음 작업에서 바뀐 점

간헐적 문제를 만나면 먼저 런타임 Inspector·장치 상태·project setting·prefab default를 함께 확인합니다. "**코드가 같으면 동작도 같다**"는 가정을 줄이고, 엔진 설정과 디바이스 상태를 체크리스트에 넣었습니다.

얻은 것

재현이 흔들리는 버그는 코드 diff만 볼 것이 아니라 **엔진 상태·입력 장치·실행 환경**까지 관찰해야 합니다. 같은 소스라도 런타임에서 선택된 장치와 scheme이 다르면 완전히 다른 결과가 나올 수 있었습니다.



▶ PLAY (WebGL)
브라우저 플레이
nullma.itch.io/cubika ↗

Shorts 영상 · youtube.com/shorts/A6-j0cwDtW4 ↗

CUBIKA

작게 완성하고 WebGL로 배포

원형 병합이 아니라 **정사각 큐브의 회전·낙하·밀치기**로 플레이 감각을 바꾼 병합 퍼즐입니다.

1주 내 솔로 완성 후 itch.io에 WebGL 배포, 이후 모바일 조작·수익화 scaffold 보강.

메커닉 — 정형화를 깨는 변수

고정 방향 낙하의 정형화를 깨려고 낙하 전 **회전**과 낙하 후 **밀치기**를 추가. 물리 충돌은 Unity Collider.Trigger로 구현.

문제 해결 — 즉시 종료 완화

game over가 너무 즉시 발생해 납득이 어려워, over-line cube가 **3초 grace** 지속될 때만 종료되도록 countdown feedback을 추가.

출시 후 보강 — 단순 업로드에서 운영 항목으로

MainMenu, pause popup, confirmation dialog, mobile controls, Unity Ads / IAP scaffold를 추가해 "완성 후의 운영"을 점검했습니다. 큐브·UI는 톤앤매너를 맞추려 Aseprite로 직접 제작했습니다.

< 1주

아이디어 → 완성 (솔로)

3초

game over grace

WebGL

itch.io 배포

얻은 것

작은 게임을 끝까지 완성하면 새 기능보다 **입력·피드백·일시정지·확인 팝업** 같은 **사용성 보강**이 더 중요해진다는 점, 배포 후의 작업에서 확인했습니다.

OTHER WORK

기타 구현 경험

Dong_Mae는 팀 프로젝트에서 레벨 기믹과 연출 구현 폭을, MyTokenMate는 개발 중 반복되던 AI 사용량 확인 문제를 **local-only 도구**로 푼 사례입니다.

메트로이드바니아 · 팀 프로젝트

Dong_Mae

레벨 기믹과 연출 구현 폭을 확보한 프로젝트입니다. 타일맵 기반 레벨에서 다양한 상호작용과 카메라 연출을 담당했습니다.

담당 구현

레벨 기믹

아이템·퍼즐·트랩 도어, 로프, 점프패드, 사다리, 이동 플랫폼, 함정.

탐색 요소

수중 구간, 파괴 가능한 숨겨진 벽 등 메트로이드바니아형 탐색.

연출

카메라 전환, fade, 보스 레터박스 컷신 연출.

레벨 디자인

보스 연출

카메라/fade

개발 보조 도구 · Electron

MyTokenMate

작업 중 반복되던 AI coding tool 사용량 확인을, API key나 네트워크 요청 없이 해결한 Electron 오버레이입니다.

설계 방향

대상

Claude Code · Codex · Gemini CLI · Copilot의 사용량.

방식

각 도구가 남기는 로컬 로그를 직접 파싱해 상태를 표시.

원칙

API key·네트워크 요청 없이 local-only로 동작.

Local log parsing

no network

Electron

HOW I WORK

작업 방식

프로젝트마다 형테는 달라도 반복되는 기준은 같습니다. 먼저 **목적과 경계**를 정하고, 구현 후에는 테스트·영상·실제 화면·운영 문서로 결과를 다시 확인합니다.

01

목적 정의

왜 만들고, 어떤 결과가 완료 기준인지 먼저 정함. Ghost March는 출시, Match3는 구조 설계, LostCone은 팬층 피드백.

02

구조 분리

asmdef·EventBus·interface·namespace로 변경 영향 범위를 작게 만들.

03

검증 분리

테스트 통과·성능 수치·영상·visual QA·플레이테스트를 서로 다른 근거로 분리.

04

회고 반영

문제를 해결하고 끝내지 않고, 다음 프로젝트의 설계 기준으로 옮김.

프로젝트별 적용

- **Ghost March** — 출시 기준본을 보존하고 DOTS 실험본과 수치 비교.
- **Match3LogueLike** — AI 작업 전에 모듈 경계와 금지 규칙을 정의.
- **IronSarcophagus** — 본인 담당과 팀원 담당 시스템을 문서로 분리.

트러블슈팅 기준

- **LostCone** — 코드를 고치기 전 Control Scheme·장치 상태를 확인.
- **Cubika** — 즉시 game over 대신 feedback-grace로 납득 가능한 실패 상태 구성.
- **Ghost March** — 최신 렌더링 API보다 프로젝트 렌더 파이프라인과 맞는 선택을 우선.

공통적으로 성능 개선은 기술 교체 자체보다 기준 장면·측정 항목·되돌릴 수 있는 범위를 먼저 정하는 일이라고 보고, 변경의 효과와 위험을 같은 자리에서 비교합니다.

LINKS & CONTACT

링크 / 연락처

링크를 누르면 브라우저에서 영상·플레이·기록으로 이동합니다.

웹 포트폴리오 (요약본)
nulma.github.io ↗

Ghost March · Google Play
앱 출시 페이지 ↗

IronSarcophagus · Video
팀 프로젝트 영상 ↗

GitHub
github.com/NulMa ↗

Ghost March · Video
플레이 영상 ↗

LostCone · Video
플레이 영상 ↗

Email
jungwt524@gmail.com ↗

Match3LogueLike · Video
플레이 영상 ↗

Cubika · itch.io
WebGL 플레이 ↗